

Protocol checker

A scoreboard asks whether the data was correct, expect versus actual

Checker vs scoreboard

- The scoreboard compares transactions, did we get the byte we expected, in the right order?
- The protocol checker enforces bus rules
- A legal beat can still carry wrong data; correct data on an illegal beat is still a bug
- In UVM you often bind SVA or a passive checker component alongside the monitor
- Monitor feeds the scoreboard; checker or assertions guard the wire protocol
- Keep the two roles separate in your head and in your testbench hierarchy

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Protocol checker sketch

INTERACTIVE TOOL

Protocol checker sketch

Contrast **bus-rule checks** with a **scoreboard**: the checker is passive and flags handshake / stability violations — not payload expect/actual. Starter: clean valid/ready beat with stable data — PASS.

Starter example: clean handshake beat (valid=1, ready=1, data 0xA5) — all protocol rules PASS.

Load starter example

Challenges 0 / 22 cleared

Quiz: role: A protocol checker's main job is to...

☐ flag bus-rule / handshake violations (usually passively)

☐ replace the scoreboard for payload compares

☐ synthesize the DUT

☐ set +UVM_TESTNAME

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

checker
Passive bus-rule watch — legal vs illegal cycles.

scoreboard
Expect vs actual on transactions / payload.

handshake
valid/ready defines a beat; hold while stalled.

separate jobs
Legal bus ≠ correct data — check both.

Lab

Protocol checker lab starter

Real UVM literacy

```
ws1 - module20-protocol-checker/examples/checker-sketch

# Track A - real Unix
# real shell session (Track A - protocol checker)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module20-protocol-checker

$ ls examples/checker-sketch/
checker.txt

$ cat examples/checker-sketch/checker.txt
Protocol checker sketch (literacy)

Two different questions:
protocol checker  → "Was the bus legal?"
scoreboard       → "Was the data correct?"

Checker role:
passive - watches pins / monitor samples each cycle
does NOT drive the DUT
flags RULE violations (handshake, stability, X, ...)

Scoreboard role:
compares expect vs actual transactions
payload, ordering, side effects
usually fed by monitor analysis_port

Example valid/ready rules:
HS - do not drop valid while stalled waiting for ready
HOLD - while valid && !ready, data must stay stable
X - when valid is high, data must be known (not X/2)

Starter beat (all PASS):
valid=1, ready=1, data=0xA5, stable → HS/HOLD/X pass
scoreboard may still compare expect 0xA5 vs actual 0xA5 separately

Typical UVM placement:
SVA bound to interface, or
passive uvm_component sampling vif each cycle, or
protocol checker inside passive agent

monitor → analysis_port → scoreboard (functional)
checker → uvm_error on rule break (protocol)

Common mistakes:
scoreboard expected to catch handshake drops
checker drives pins (should be passive)
checking during reset when signals are meaningless
one passing rule assumed to mean all rules pass

Legacy reference:
learn_uvm2017_sv_verilator/module6/examples/protocol_checkers/protocol_checkers.sv

$ grep -n "CheckerComp\|valid\|ready\|uvm_error"
../learn_uvm2017_sv_verilator/module6/examples/protocol_checkers/protocol_checkers.sv | head -114
19: * - Detect protocol events (valid asserted)
20: * - Check protocol rules (valid held until ready)
35: * - valid: Valid signal (source asserts when data valid)
36: * - ready: Ready signal (destination asserts when ready)
40: logic valid; // Valid signal
41: logic ready; // Ready signal
48: * CheckerComp: Protocol checker component
56: * - When valid is asserted, it should remain asserted until ready
57: * - Valid should not be held without ready for too long (timeout)
60: class CheckerComp extends uvm_component;
63:   uvm_component_utils(CheckerComp)
109:   // - When valid is asserted, it should remain asserted until ready
110:   // - Valid should not be held without ready for too long
112:   if (vif.valid && !vif.ready) begin
```

Real shell — protocol checker sketch

Pitfalls to watch

- Do not expect the scoreboard to catch dropped valid
- Do not drive pins from the checker, it should be passive
- Do not disable checking during reset without meaning to, rules apply once reset releases
- Do not conflate one passing rule with all rules passing, check each handshake, hold
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, pass the starter beat, then break one rule and name which checker fired
- On real UVM, sketch three valid-ready rules and one scoreboard comparison
- When you are ready, take the short quiz, then continue to VIP anatomy in the next module